Dynamic Attack Graphs: Incremental Re-computation with Differential Dataflow

Stefania-Cristina Mozacu Emil C. Lupu

Department / Affiliation

May 2026

Dynamic Attack Graphs

2026-05-11

Dynamic Attack Graphs: Incremental Re-computation with Differential Dataflow

Stefania-Cristina Mozacu Emil C. Lupu

Department / Affiliation

May 2026

This presentation summarises a proof-of-concept implementation of dynamic attack graphs using differential dataflow.

Executive summary

- Problem: full recomputation of attack graphs is costly for live networks
- Approach: use differential dataflow (Timely) to incrementally update derived facts
- Artifacts: Rust implementation, benchmarks (chain/star/random cut), Graphviz visualisations
- Results: large speedups on star/topologies; position-dependent cost on chain

Dynamic Attack Graphs

Executive summary

Executive summary

- Problem: full recomputation of attack graphs is costly for live networks
- Approach: use differential dataflow (Timely) to incrementally update derived facts
- Artifacts: Rust implementation, benchmarks (chain/star/random cut), Graphviz visualisations
- Results: large speedups on star/topologies; position-dependent cost on chain

Give a brief overview: problem, technique, implementation, and headline results.

Motivation

- Attack graphs summarise attacker reachability using facts and rules
- Modern networks are dynamic: CVEs, patches, topology changes
- Recomputing the entire graph on every change is impractical at scale

Dynamic Attack Graphs

Motivation

Motivation

- Attack graphs summarise attacker reachability using facts and rules
- Modern networks are dynamic: CVEs, patches, topology changes
- Recomputing the entire graph on every change is impractical at scale

Give examples where rapid updates are needed: patch management, red-team, live monitoring.

Background: attack graphs and MulVAL

- Attack graphs: facts (vulnerabilities, network edges, firewall rules) and inference rules
- MulVAL uses Datalog-like rules to compute derived reachability facts
- Example rule (informal): execCode(A,D,P) :- execCode(A,Src,$_{)}$, *effectiveAccess*(*Src*, *D*, *Svc*), *vulnerability*(*D*,, *Svc*, *P*)

Background: attack graphs and MulVAL

- Attack graphs: facts (vulnerabilities, network edges, firewall rules) and inference rules
- MulVAL uses Datalog-like rules to compute derived reachability facts
- Example rule (informal): execCode(A,D,P) :- execCode(A,Src$_,$), $effectiveAccess(Src, D, Svc)$, $vulnerability(D_{,}, Svc, P)$

Explain the Datalog style rules and how they map to joins and recursive definitions.

Background: differential dataflow

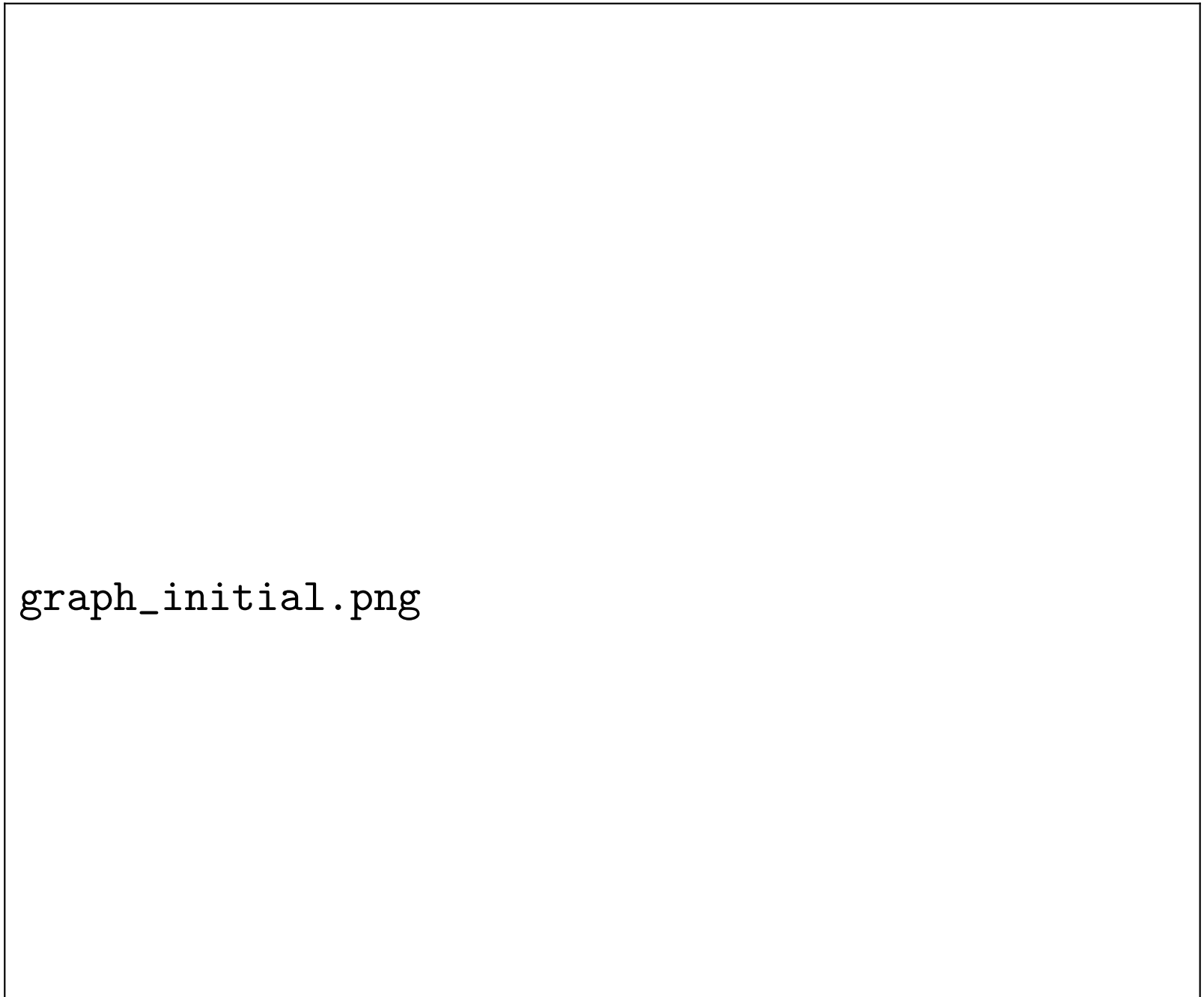
- Differential dataflow propagates deltas and supports iterative fixed-point computation
- Operators: join, antijoin, iterate, distinct, consolidate
- Natural fit for incremental Datalog: update deltas, recompute only affected facts

Background: differential dataflow

- Differential dataflow propagates deltas and supports iterative fixed-point computation
- Operators: join, antijoin, iterate, distinct, consolidate
- Natural fit for incremental Datalog: update deltas, recompute only affected facts

Highlight why iterate() + distinct() yields termination and how antijoin handles negation (firewall denies).

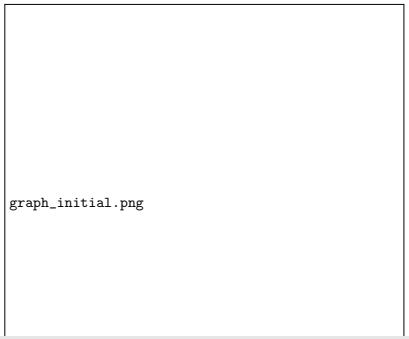
System overview



Dynamic Attack Graphs

System overview

System overview



High-level pipeline: inputs (vulns, network, firewall, positions, goals) flow into the differential pipeline producing execCode, ownsMachine, goalsReached.

Data model (facts)

- Input facts: `VulnerabilityRecord(host, cve, service, privilege)`
- `NetworkAccessRule(src, dst, service)`
- `FirewallRuleRecord(src, dst, service, action)`
- `AttackerStartingPosition(attacker, host, privilege)`
- `AttackerTargetGoal(attacker, target)`

Dynamic Attack Graphs

Data model (facts)

Data model (facts)

- Input facts: `VulnerabilityRecord(host, cve, service, privilege)`
- `NetworkAccessRule(src, dst, service)`
- `FirewallRuleRecord(src, dst, service, action)`
- `AttackerStartingPosition(attacker, host, privilege)`
- `AttackerTargetGoal(attacker, target)`

Walk through the shapes of these records and the rationale for fields.

Derived facts and rules

- `EffectiveNetworkAccess(src,dst,svc)` = network **ANTIJOIN** denies
- `AttackerCodeExecution(attacker,host,priv)` formed by recursion via `iterate()`
- `AttackerOwnsMachine(attacker,host)` when `priv==Root`
- `AttackerGoalReached(attacker,target)` via semijoin on owned machines

Dynamic Attack Graphs

Derived facts and rules

Derived facts and rules

- `EffectiveNetworkAccess(src,dst,svc)` = network **ANTIJOIN** denies
- `AttackerCodeExecution(attacker,host,priv)` formed by recursion via `iterate()`
- `AttackerOwnsMachine(attacker,host)` when `priv==Root`
- `AttackerGoalReached(attacker,target)` via semijoin on owned machines

Explicitly connect each derived fact to the operator chain used in the code.

Translating rules to dataflow (example)

- effectiveAccess = network$_{r}$*ules*.*antijoin*(*deny*$_{k}$*eys*)
- execCode: iterate over current execCode; join with access (by src); join with vulnerabilities (by host,service); concat + distinct

Translating rules to dataflow (example)

Translating rules to dataflow (example)

- effectiveAccess = network$_{r}$*ules*.*antijoin*(*deny*$_{k}$*eys*)
- execCode: iterate over current execCode; join with access (by src); join with vulnerabilities (by host,service); concat + distinct

Show how join keys are chosen and why distinct() is necessary to ensure termination.

Implementation details

- Language: Rust
- Libraries: timely, differential$_{d}$*ataflow*, *abomonation*(*fasttransfer*)

- Important functions: `build`$_{a}$*ttack*$_{g}$*raph*,
- `Single-worker benchmarks via timely::execute`$_{d}$*irectlyforreproducibility*

Dynamic Attack Graphs

Implementation details

Implementation details

- Language: Rust
- Libraries: timely, differential$_{d}$*ataflow*, *abomonation*(*fasttransfer*)
- Important functions: `build`$_{a}$*ttack*$_{g}$*raph*,
- `Single-worker benchmarks via timely::execute`$_{d}$*irectlyforreproducibility*

Mention reasons for Rust choice (performance, control) and abomonation for serialization performance across workers.

Visualization and export

- Graph export: DOT nodes/edges from derived facts
- Use Graphviz to render: `dot -Tpng` graph_initial.dot − ograph_initial.png

- `Visual conventions: compromised hosts, attacker start, goal highlighted`

Dynamic Attack Graphs

Visualization and export

Visualization and export

- Graph export: DOT nodes/edges from derived facts
- Use Graphviz to render: `dot -Tpng graph_initial.dot -o graph_initial.png`
- `Visual conventions: compromised hosts, attacker start, goal highlighted`

Explain how the exported graphs are used to illustrate the effect of patches.

Benchmarks: setup

- Topologies: chain, star, mesh; synthetic vulnerabilities
- Metrics: initial full recomputation time, incremental update time, speedup
- Repeatable: deterministic generators, fixed seeds, single-worker execution

Dynamic Attack Graphs

Benchmarks: setup

Benchmarks: setup

- Topologies: chain, star, mesh; synthetic vulnerabilities
- Metrics: initial full recomputation time, incremental update time, speedup
- Repeatable: deterministic generators, fixed seeds, single-worker execution

Explain why each topology was chosen to highlight different behaviors.

Star topology: results (example)

Nodes	Initial (ms)	Incremental (ms)	Speedup
100	12.1	0.02	605x
1000	39.0	0.13	300x
5000	210	0.9	233x

Table 1: Sample star topology timings (replace with measured numbers).

Star topology: results (example)

Nodes	Initial (ms)	Incremental (ms)	Speedup
100	12.1	0.02	605×
1000	39.0	0.13	300×
5000	210	0.9	233×

Table 1: Sample star topology timings (replace with measured numbers).

Interpretation: incremental updates are effectively constant time due to low depth.

Chain topology: results (example)

Nodes	Initial (ms)	Incremental (ms)	Speedup
100	9.0	4.2	2.1x
1000	95.0	62.0	1.5x
5000	450	400	1.1x

Table 2: Sample chain topology timings (replace with measured numbers).

Dynamic Attack Graphs

Chain topology: results (example)

Chain topology: results (example)

Nodes	Initial (ms)	Incremental (ms)	Speedup
100	9.0	4.2	2.1x
1000	95.0	62.0	1.5x
5000	450	400	1.1x

Table 2: Sample chain topology timings (replace with measured numbers).

Interpretation: chain is worst-case for cuts near the head.

Random-cut experiment (chain)

Dynamic Attack Graphs

Random-cut experiment (chain)

Random-cut experiment (chain)

Scatter plot of incremental update time vs cut position: cost drops as cut position moves towards chain end.

Statistical analysis

- RQ1: median speedup across topologies (report numbers)
- RQ2: regression of initial time vs N (O(N) observed) and incremental vs N (depend on topology)
- RQ3: cut position strongly correlates with incremental cost (report R^2)

Statistical analysis

- RQ1: median speedup across topologies (report numbers)
- RQ2: regression of initial time vs N (O(N) observed) and incremental vs N (depend on topology)
- RQ3: cut position strongly correlates with incremental cost (report R^2)

Include brief statistical method notes (paired tests, regression models).

Discussion

- Incremental approach highly effective for shallow, star-like topologies
- Less beneficial for deep chain-like dependencies or global changes
- Suggest hybrid strategies for real deployments

Dynamic Attack Graphs

Discussion

- Incremental approach highly effective for shallow, star-like topologies
- Less beneficial for deep chain-like dependencies or global changes
- Suggest hybrid strategies for real deployments

Link to operational implications: alert triage, live patching, selective re-computation.

Limitations and future work

- Synthetic topologies; need real network datasets
- Single-worker benchmarks; evaluate multi-worker scaling
- Extend vulnerability modelling and integrate scanners

Dynamic Attack Graphs

Limitations and future work

- Synthetic topologies; need real network datasets
- Single-worker benchmarks; evaluate multi-worker scaling
- Extend vulnerability modelling and integrate scanners

Propose immediate next steps and potential thesis work.

Reproducibility and commands

- Build: `cargo build --release`
- Run benchmarks: `cargo run --release --bin attack-graph`
- Render graphs: `dot -Tpng graph_initial.dot – ograph_initial.png`

- `Docker: docker build -t attack-graph . docker run --rm attack-graph`

Reproducibility and commands

- Build: `cargo build --release`
- Run benchmarks: `cargo run --release --bin attack-graph`
- Render graphs: `dot -Tpng graph_initial.dot – ograph_initial.png`

- Docker: `docker build -t attack-graph .` `docker run --rm attack-graph`

List file locations: `src/benchmarks.rs`, `src/rules.rs`, `graph_initial.dot`.

Demo visual walkthrough

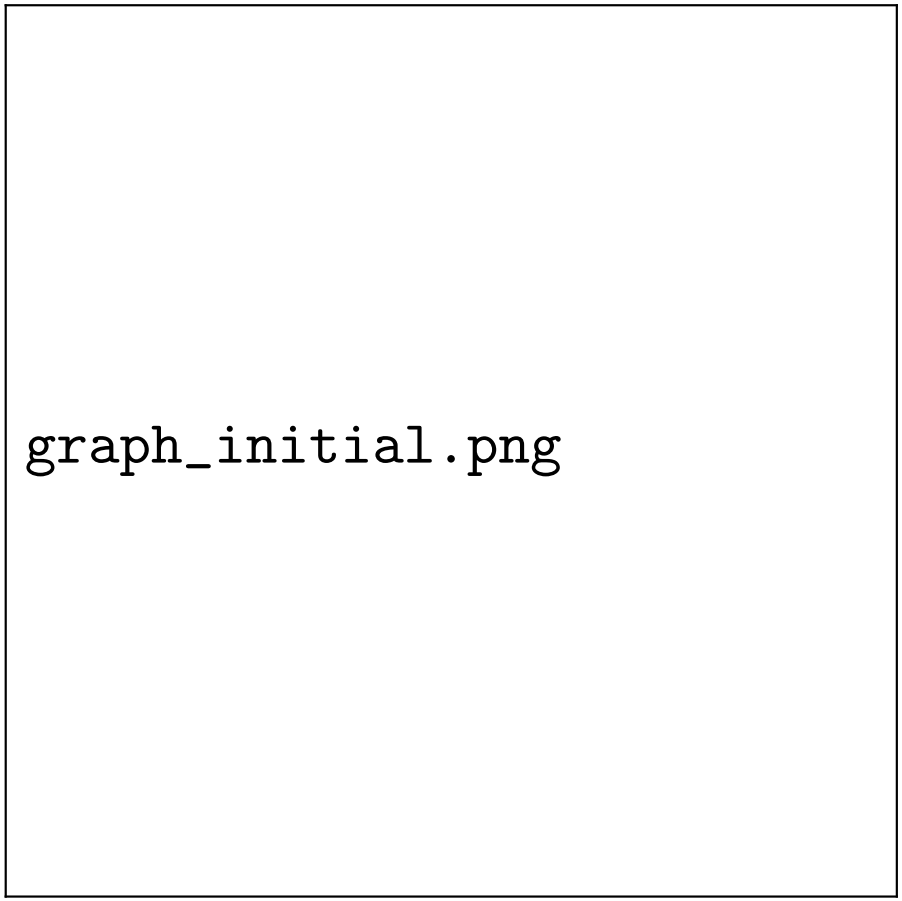


Figure 1: Before patch

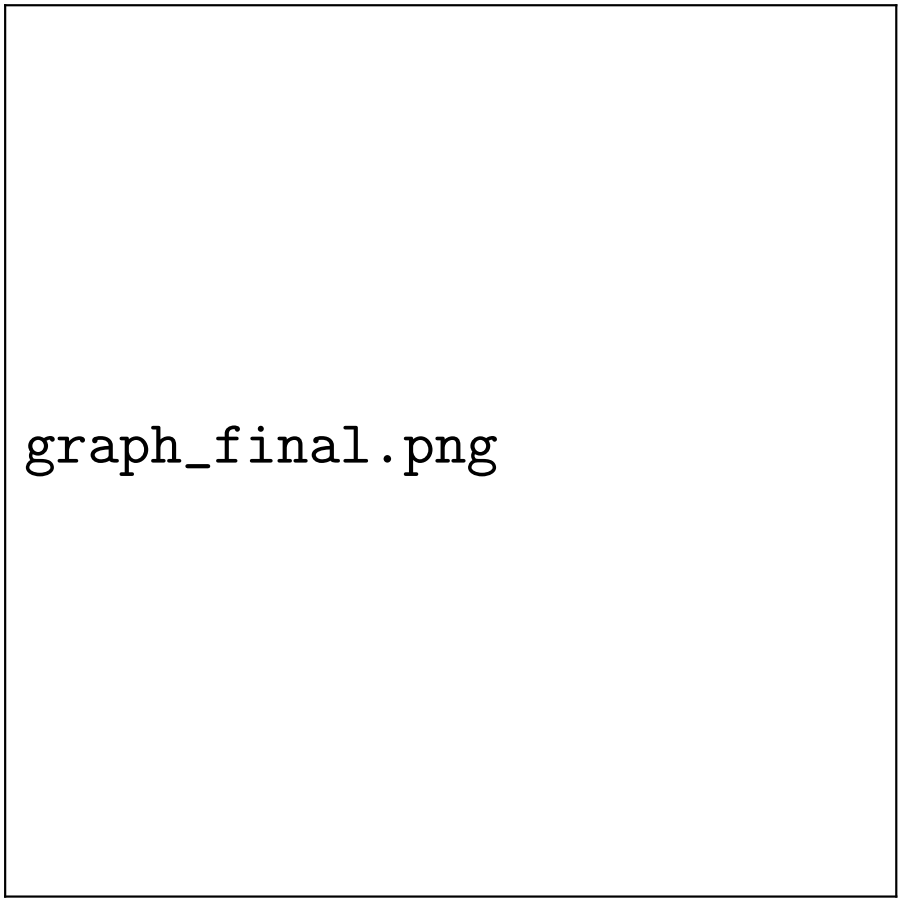


Figure 2: After patch

Dynamic Attack Graphs

Demo visual walkthrough

Demo visual walkthrough

graph_initial.png

Figure 1: Before patch

graph_final.png

Figure 2: After patch

Describe the visual change: removed attack path, nodes recoloured to show compromised hosts.

Related work

- MulVAL and Datalog-based attack graphs
- Differential Dataflow (Naiad lineage) and incremental Datalog work
- Incremental program analysis and streaming graph processing

Dynamic Attack Graphs

Related work

Related work

- MulVAL and Datalog-based attack graphs
- Differential Dataflow (Naiad lineage) and incremental Datalog work
- Incremental program analysis and streaming graph processing

Position our contributions against these works.

Conclusion

- Demonstrated incremental recomputation with differential dataflow for attack graphs
- Showed large speedups in common topologies and characterized worst-cases
- Path forward: multi-worker scaling, real datasets, CVE semantics

Dynamic Attack Graphs

Conclusion

Concise closing remarks and invitation for questions.

Conclusion

- Demonstrated incremental recomputation with differential dataflow for attack graphs
- Showed large speedups in common topologies and characterized worst-cases
- Path forward: multi-worker scaling, real datasets, CVE semantics

Appendix: exact commands and file locations

`Run benchmarks (example) cargo run --release --bin attack-graph`
`Render graphs dot -Tpng graph¡nitial.dot – ograph¡nitial.png`
`Docker docker build -t attack-graph . docker run --rm attack-graph`

Dynamic Attack Graphs

Appendix: exact commands and file locations

Appendix: exact commands and file locations

```
Run benchmarks (example) cargo run --release --bin attack-graph
Render graphs dot -Tpng graph/nitial.dot – ograph/nitial.png
Docker docker build -t attack-graph . docker run --rm attack-graph
```

Provide GitHub repo URL and pointer to Dockerfile in project root.